

Counting Operations

We turn to counting the flops required to solve a square linear system $Ax = b$ that has a unique solution. We assume that no row interchanges are necessary, which is frequently the case.

Suppose that we solve the system $Ax = b$, using the Gauss method with back substitution. Form the augmented matrix

$$[A | b] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ & & \ddots & & \vdots \\ & & & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right].$$

For the moment let us neglect the flops performed on b and count just the flops performed on A . In reducing the $n \times n$ matrix A to upper-triangular form U , we execute $n - 1$ flops in adding a multiple of the first row of $[A | b]$ to the second row. (We do not need to compute the zero entry at the beginning of our new second row; we know it will be zero.) We similarly use $n - 1$ flops to obtain the new row 3, and so on. This gives a total of $(n - 1)^2$ flops executed using the pivot in row 1. The pivot in row 2 is then used for the $(n - 1) \times (n - 1)$ matrix obtained by crossing off the first row and column of the modified coefficient matrix. By the count just made for the $n \times n$ matrix, we realize that using this pivot in the second row will result in execution of $(n - 2)^2$ flops. Continuing in this fashion, we see that reducing A to upper-triangular form U will require

$$(n - 1)^2 + (n - 2)^2 + (n - 3)^2 + \cdots + 1 \quad (5)$$

flops, together with some divisions. Let's count the divisions. We expect to use just one division each time a row is multiplied by a constant and added to another row (see point 2). There will be $n - 1$ divisions involving the pivot in row 1, $n - 2$ involving the pivot in row 2, and so on, for a total of

$$(n - 1) + (n - 2) + (n - 3) + \cdots + 1 \quad (6)$$

divisions.

There are some handy formulas for finding a sum of consecutive integers and a sum of squares of consecutive integers. It can be shown by induction (see Appendix A) that

$$1 + 2 + 3 + \cdots + n = \frac{n(n + 1)}{2} \quad (7)$$

and

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n + 1)(2n + 1)}{6}. \quad (8)$$

Replacing n by $n - 1$ in formula (8), we see that the number of flops given by sum (5) is equal to

$$\begin{aligned}\frac{(n-1)n(2n-1)}{6} &= \frac{(n^2-n)(2n-1)}{6} = \frac{1}{6}(2n^3 - 3n^2 + n) \\ &= \frac{n^3}{3} - \frac{n^2}{2} + \frac{n}{6}.\end{aligned}\quad (9)$$

We assume that we are using a computer to solve a *large* linear system involving hundreds or thousands of equations. With $n = 1000$, the value of Eq. (9) becomes

$$\frac{1,000,000,000}{3} - \frac{1,000,000}{2} + \frac{1000}{6}.\quad (10)$$

The largest term in expression (10) is the first one, $1,000,000,000/3$, corresponding to the $n^3/3$ term in Eq. (9). The lower powers of n in the $n^2/2$ and $n/6$ terms contribute little in comparison with the $n^3/3$ term for large n . It is customary to keep just the $n^3/3$ term as a measure of the **order of magnitude** of the expression in Eq. (9) for large values of n .

Turning to the count of the divisions in sum (6), we see from Eq. (7) with n replaced by $n - 1$ that we are using

$$\frac{(n-1)n}{2} = \frac{n^2}{2} - \frac{n}{2}\quad (11)$$

divisions. For large n , this is of order of magnitude $n^2/2$, which is inconsequential in comparison with the order of magnitude $n^3/3$ for the flops. Exercise 1 shows that the number of flops performed on the column vector $\mathbf{b}$ in reducing $[A | \mathbf{b}]$ is again given by Eq. (11), so it can be neglected for large n in view of the order of magnitude $n^3/3$. The result is shown in the following box.

Flop Count for Reducing $[A | \mathbf{b}]$ to $[U | \mathbf{c}]$

If $A\mathbf{x} = \mathbf{b}$ is a square linear system in which A is an $n \times n$ matrix, the number of flops executed in reducing $[A | \mathbf{b}]$ to the form $[U | \mathbf{c}]$ is of order of magnitude $n^3/3$ for large n .

We turn now to finding the number of flops used in back substitution to solve the upper-triangular linear system $U\mathbf{x} = \mathbf{c}$. This system can be written out as

$$\begin{aligned}u_{11}x_1 + \cdots + u_{1,n-1}x_{n-1} + u_{1n}x_n &= c_1 \\ &\vdots \\ u_{n-1,n-1}x_{n-1} + u_{n-1,n}x_n &= c_{n-1} \\ u_{nn}x_n &= c_n.\end{aligned}$$

Solving for x_n requires one indexed division, which we consider to be a flop. Solving then for x_{n-1} requires an indexed multiplication and subtraction, followed by an indexed division, which we consider to be two flops. Solving for x_{n-2} requires two flops, each consisting of a multiplication combined with a subtraction, followed by an indexed division, and we consider this to contribute three more flops, and so on. We obtain the count

$$1 + 2 + 3 + \cdots + n = \frac{(n+1)n}{2} = \frac{n^2}{2} + \frac{n}{2} \quad (12)$$

for the flops in this back substitution. Again, this is of lower order of magnitude than the order of magnitude $n^3/3$ for the flops required to reduce $[A \mid \mathbf{b}]$ to $[U \mid \mathbf{c}]$.

Flop Count for Back Substitution

If U is an $n \times n$ upper-triangular matrix, then back substitution to solve $U\mathbf{x} = \mathbf{c}$ requires the order of magnitude $n^2/2$ flops for large n .

Combining the results shown in the last two boxes, we arrive at the following flop count.

Flop Count for Solving $A\mathbf{x} = \mathbf{b}$, Using the Gauss Method with Back Substitution

If A is an $n \times n$ matrix, the number of flops required to solve the system $A\mathbf{x} = \mathbf{b}$ using the Gauss method with back substitution is of order of magnitude $n^3/3$ for large n .

EXAMPLE 1 For the computer that produced the execution times shown in Table 10.1, find the approximate time required to solve a system of 100 equations in 100 unknowns, using single-precision arithmetic and (a) interpretive BASIC, (b) compiled BASIC.

SOLUTION From the flop count for the Gauss method, we see that solving such a system with $n = 100$ requires about $100^3/3 = 1,000,000/3$ flops. In interpretive BASIC, the time required for 10,000 flops in single precision was about 143 seconds. Thus the $1,000,000/3$ flops require about $(1,000,000/30,000)143 = 14,300/3$ seconds, or about 1 hour and 20 minutes. In compiled BASIC, where 10,000 flops took about 15 seconds, we find that the time is approximately $(1,000,000/30,000)15 = 1500/3 = 500$ seconds, or about 8 minutes. The PC used for Table 10.1 is regarded today as terribly slow. ■

It is interesting to compare the efficiency of the Gauss method with back substitution to that of the Gauss-Jordan method. Recall that, in the Gauss-Jordan method, $[A | b]$ is reduced to a form $[I | c]$, where I is the identity matrix. Exercise 2 shows that Gauss-Jordan flop count is of order of magnitude $n^3/2$ for large n . Thus the Gauss-Jordan method is not as efficient as the Gauss method with back substitution if n is large. One expects a Gauss-Jordan program to take about one and a half times as long to execute. The routine TIMING in LINTEK can be used to illustrate this. (See Exercises 22 and 24.)

Counting Flops for Matrix Operations

The exercises ask us to count the flops involved in matrix addition, multiplication, and exponentiation. Recall that a matrix product $C = AB$ is obtained by taking dot products of row vectors of an $m \times n$ matrix A with column vectors of an $n \times s$ matrix B . We indicate the usual way that a computer finds the dot product c_{ij} in C . First the computer sets $c_{ij} = 0$. Then it replaces c_{ij} by $c_{ij} + a_{i1}b_{1j}$, which gives c_{ij} the value $a_{i1}b_{1j}$. Then the computer replaces c_{ij} by $c_{ij} + a_{i2}b_{2j}$, giving c_{ij} the value $a_{i1}b_{1j} + a_{i2}b_{2j}$. This process continues in the obvious way until finally we have accumulated the desired value,

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}.$$

Each of these replacements of c_{ij} is accomplished by means of a flop, typically expressed by

$$C(I,J) = C(I,J) + A(I,K) * B(K,J) \quad (13)$$

in BASIC.

EXAMPLE 2 Find the number of flops required to compute the dot product of two vectors, each with n components.

SOLUTION The preceding discussion shows that the dot product uses n flops of form (13), corresponding to the values 1, 2, 3, ..., n for K . ■

SUMMARY

1. A flop is a rather vaguely defined execution by a computer, consisting typically of a bit of indexing, the retrieval and storage of a couple of values, and one or two arithmetic operations. Typical flops might appear in a computer program in instruction lines such as

$$C(I,J) = A(I,J) + B(I,J)$$

or

$$A(I,J) = A(I,J) + R * A(K,J).$$

2. A computer takes time to execute a flop, and it is desirable to use as few flops as possible when performing an extensive computation.
3. If the number of flops used to solve a problem is given by a polynomial expression in n , it is customary to keep only the term of highest degree in the polynomial as a measure of the *order of magnitude* of the number of flops when n is large.
4. The order of magnitude of the number of flops used in solving a system $Ax = b$ for an $n \times n$ matrix A is

$$\frac{n^3}{3} \text{ for the Gauss method with back substitution}$$

and

$$\frac{n^3}{2} \text{ for the Gauss-Jordan method.}$$

5. The formulas

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

and

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

are handy for determining the number of flops performed by a computer in matrix computations.

EXERCISES

all of these exercises, assume that no row
ectors are interchanged in a matrix.

1. Let A be an $n \times n$ matrix. Show that, in reducing $[A | b]$ to $[U | c]$ using the Gauss method, the number of flops performed on b is of order of magnitude $n^2/2$ for large n .
2. Let A be an $n \times n$ matrix. Show that, in solving $Ax = b$ using the Gauss-Jordan method, the number of flops has order of magnitude $n^3/2$ for large n .

Exercises 3–11, let A be an $n \times n$ matrix, and let B and C be $m \times n$ matrices. For each matrix, find the number of flops required for efficient computation of the matrix.

- | | | |
|------------|--------------|--------------|
| 3. $B + C$ | 4. A^2 | 5. BA |
| 6. A^3 | 7. A^4 | 8. A^5 |
| 9. A^6 | 10. A^{63} | 11. A^{64} |

12. Which of the following is more efficient with a computer?
 - a. Solving a square system $Ax = b$ by the Gauss-Jordan method, which makes each pivot 1 before creating the zeros in the column containing it.
 - b. Using a similar technique to reduce the system to a diagonal system $Dx = c$, where the entries in D are not necessarily all 1, and then dividing by these entries to obtain the solution.

13. Let A be an $n \times n$ matrix, where n is large. Find the order of magnitude for the number of flops if A^{-1} is computed using the Gauss–Jordan method on the augmented matrix $[A \mid I]$ without trying to reduce the number of flops used on I in response to the zeros that appear in it.
14. Repeat Exercise 13, using the Gauss method with back substitution rather than the Gauss–Jordan method.
15. Repeat Exercise 14, but this time cut down the number of flops performed on I during the Gauss reduction by taking into account the zeros in I .

Exercises 16–20 concern band matrices. In a number of situations, square linear systems $Ax = b$ occur in which the nonzero entries in the $n \times n$ matrix A are all concentrated near the main diagonal, running from the upper left-hand corner to the lower right-hand corner of A . Such a matrix is called a **band matrix**. For example, the matrix

$$\begin{array}{c}
 w = 2 \\
 \left[\begin{array}{cccccc}
 2 & 1 & 0 & 0 & 0 & 0 \\
 1 & 3 & 4 & 0 & 0 & 0 \\
 0 & 4 & 1 & 2 & 0 & 0 \\
 0 & 0 & 2 & 2 & 7 & 0 \\
 0 & 0 & 0 & 7 & 1 & 3 \\
 0 & 0 & 0 & 0 & 3 & 5
 \end{array} \right] \\
 w = 2
 \end{array} \quad (14)$$

is a symmetric 6×6 band matrix. We say that the **band width** of a band matrix $[a_{ij}]$ is w if w is the smallest integer such that $a_{ij} = 0$ for $|i - j| \geq w$. Thus matrix (14) has band width $w = 2$, as indicated. Such a matrix of band width 2 is also called **tridiagonal**. We usually assume that the band width is small compared with the size of n . As the band width approaches n , the matrix becomes full.

In Exercises 16–20, assume that A is an $n \times n$ band matrix with band width w that is small in comparison with n .

16. What can be said concerning the band width of A^2 ? of A^3 ? of A^m ?
17. Let A be tridiagonal, so $w = 2$. Find the order of magnitude of the number of flops required to reduce the partitioned matrix

$[A \mid b]$ to a form $[U \mid c]$, where U is an upper-triangular matrix, taking into account the banded character of A .

18. Repeat Exercise 17 for band width w , expressing the result in terms of w .
19. Repeat Exercise 18, but include the flops used in back substitution to solve $Ax = b$.
20. Explain why the Gauss method with back substitution is much more efficient than the Gauss–Jordan method for a banded matrix where w is very small compared with n .
21. Mark each of the following True or False.
 - a. A flop is a very precisely defined entity.
 - b. Computers can work so fast that it is not worthwhile to try to minimize the number of computations a computer makes to solve a problem, provided that the number of computations is only a few hundred.
 - c. Computers can work so fast that it is not worthwhile to try to minimize the number of computations required to solve a problem.
 - d. The Gauss method with back substitution and the Gauss–Jordan method for solving a large linear system both take about the same amount of computer time.
 - e. The Gauss–Jordan method for solving a large linear system takes about half again as long to execute as does the Gauss method on a computer.
 - f. Multiplying two $n \times n$ matrices requires more flops than does solving a linear system with an $n \times n$ coefficient matrix.
 - g. About n^2 flops are required to execute the back substitution in solving a linear system with an $n \times n$ coefficient matrix by using the Gauss method.
 - h. Executing the Gauss method with back substitution for a large linear system with an $n \times n$ coefficient matrix requires about n^2 flops.
 - i. Executing the Gauss method with back substitution for a large linear system with an $n \times n$ coefficient matrix requires about $n^3/3$ flops.
 - j. Executing the Gauss–Jordan method for a large linear system with an $n \times n$ coefficient matrix requires about $n^3/2$ flops.

LINTEK contains a routine *TIMING* that can be used to time algebraic operations and flops. The program can also be used to time the solution of square systems $Ax = b$ by the Gauss method with back substitution and by the Gauss-Jordan method. For a user-specified integer $n \leq 80$, the program generates the $n \times n$ matrix A and column vector b , where all entries are in the interval $[-20, 20]$. Use *TIMING* for Exercises 22–24.

22. Experiment with *TIMING* to see roughly how many of the indicated operations your computer can do in 5 seconds.
 - a. Additions
 - b. Subtractions
 - c. Multiplications
 - d. Divisions
 - e. Flops
23. Run the routine *TIMING* in *LINTEK* to time the Gauss method and the Gauss-Jordan method, starting with small values of n and increasing them until a few seconds' difference in times for the two methods is obtained. Does the time for the Gauss-Jordan method seem to be about $\frac{3}{2}$ the time for the Gauss method with back substitution? If not, why not?
24. Continuing Exercise 23, increase the size of n until the solutions take 2 or 3 minutes. Now does the Gauss-Jordan method seem to take about $\frac{3}{2}$ times as long? If this ratio is significantly different from that obtained in Exercise 23, explain why. (The two ratios may or may not appear to be approximately the same, depending on the speed of the computer used and on whether time in fractions of seconds is displayed.)

MATLAB

The command *clock* in *MATLAB* returns a row vector

[year, month, day, hour, minute, second]

that gives the date and time in decimal form. To calculate the elapsed time for a computation, we can

set $t0 = \text{clock}$,
execute the computation,
give the command $\text{etime}(\text{clock}, t0)$, which returns elapsed time since $t0$.

The student edition of *MATLAB* will not accept a value of i greater than 1024 in a "FOR $i = 1$ to n " loop, so to calculate the time for more than 1024 operations in a loop, we use a double

FOR $g = 1$ to h , FOR $i = 1$ to n

loop with which we can find the time required for up to 1024^2 operations. Note below that the syntax of the *MATLAB* loops is a bit different from that just displayed. Recall that the colon can be used with the meaning "through." The first two lines below define data to be used in the last line. The last line is jammed together so that it will all fit on one screen line when you modify the $c=a+b$ portion in Exercise M5 to time flops. As shown below, the last line returns the elapsed time for $h \cdot n$ repeated additions of two numbers a and b between 0 and 1:

```
A = rand(6,6); a = rand; b = rand; r = rand; j = 2; k = 4; m = 3;
h = 100; n = 50;
t0=clock;for g=1:h,for i=1:n,c=a+b;end;end;etime(clock,t0)
```

- M1. Enter in the three lines shown above. Put spaces in the last line only after "for". Using the up-arrow key and changing the value of n (and h if necessary), find roughly how many additions MATLAB can perform in 5 seconds on your computer. (Remember, the time given is for $h \cdot n$ additions.)
- M2. Modify the $c=a+b$ portion and repeat M1 for subtractions.
- M3. Modify the $c=a+b$ portion and repeat M1 for multiplications.
- M4. Modify the $c=a+b$ portion and repeat M1 for divisions.
- M5. Modify the $c=a+b$ portion and repeat M1 for flops $A(k,j)=A(k,j)+r*A(m,j)$. Delete $c=a+b$ first, and don't insert any spaces!

10.2

THE LU-FACTORIZATION

Keeping a Record of Row Operations

We continue to work with a square linear system $Ax = b$ having a unique solution that can be found by using Gauss elimination with back substitution, without having to interchange any rows. That is, the matrix A can be row-reduced to an upper-triangular matrix U , without making any row interchanges.

Situations occur in which it is necessary to solve many such systems, all having the same coefficient matrix A but different column vectors b . We solve such multiple systems by row-reducing a single augmented matrix

$$[A \mid b_1, b_2, \dots, b_s]$$

in which we line up all the different column vectors $b_1, b_2, \dots, b_s$ on the right of the partition. In practice, it may be impossible to solve all of these systems by using this single augmentation in a single computer run. Here are some possibilities in which a sequence of computer runs may be needed to solve all the systems:

1. Remember that we are concerned with *large* systems. If the number s of vectors b_j is large, there may not be room in the computer memory to accommodate all of these data at one time. Indeed, it might even be necessary to reduce the $n \times n$ matrix A in segments, if n is very large. If we can handle the s vectors b_j only in groups of m at a time, we must use at least s/m computer runs to solve all the systems.
2. Perhaps the vectors b_j are generated over a period of time, and we need to solve systems involving groups of the vectors b_j as they are generated. For example, we may want to solve $Ax = b_j$ with r different vectors b_j each day.
3. Perhaps the vector b_{j+1} depends on the solution of $Ax = b_j$. We would then have to solve $Ax = b_1$, determine b_2 , solve $Ax = b_2$, determine b_3 , and so on, until we finally solved $Ax = b_s$.

From Section 10.1, we know that the magnitude of the number of flops required to reduce A to an upper-triangular matrix U is $n^3/3$ for large n . We want to avoid having to repeat all this work done in reducing A after the first computer run.

We assume that A can be reduced to U without interchanging rows—that is, that (nonzero) pivots always appear where we want them as we reduce A to U . This means that only elementary row-addition operations (those that add a multiple of a row vector to another row vector) are used. Recall that a row-addition operation can be accomplished by multiplying on the left by an $n \times n$ elementary matrix E , where E is obtained by applying the same row-addition operation to the identity matrix. There is a sequence $E_1, E_2, \dots, E_h$ of such elementary matrices such that

$$E_h E_{h-1} \cdots E_2 E_1 A = U. \quad (1)$$

Once the matrix U has been found by the computer, the data in it can be stored on a disk or on tape, and simply read in for future computer runs. However, when we are solving $Ax = b$ by reducing the augmented matrix $[A | b]$, the sequence of elementary row operations described by $E_h E_{h-1} \cdots E_2 E_1$ in Eq. (1) must be applied to the *entire* augmented matrix, not merely to A . Thus we need to keep a *record* of this sequence of row operations to perform on column vectors b , when they are used in subsequent computer runs.

Here is a way of recording the row-addition operations that is both efficient and algebraically interesting. As we make the reduction of A to U , we create a *lower-triangular matrix* L , which is a record of the row-addition operations performed. We start with the $n \times n$ identity matrix I , and as each row-addition operation on A is performed, we change one of the zero entries below the diagonal in I to produce a record of that operation. For example, if during the reduction we add 4 times row 2 to row 6, we place -4 in the *second* column position in the *sixth* row of the matrix L that we are creating as a record. The general formulation is shown in the following box.

Creation of the Matrix L

Start with the $n \times n$ identity matrix I . If during the reduction of A to U , r times row i is added to row k , replace the zero in row k and column i of the identity matrix by $-r$. The final result obtained from the identity matrix is L .

EXAMPLE 1 Reduce the matrix

$$A = \begin{bmatrix} 1 & 3 & -1 \\ 2 & 8 & 4 \\ -1 & 3 & 4 \end{bmatrix}$$

to upper-triangular form U , and create the matrix L described in the preceding box.

SOLUTION We proceed in two columns, as follows:

Reduction of A to U

$$A = \begin{bmatrix} 1 & 3 & -1 \\ 2 & 8 & 4 \\ -1 & 3 & 4 \end{bmatrix}$$

$$\sim \begin{bmatrix} 1 & 3 & -1 \\ 0 & 2 & 6 \\ -1 & 3 & 4 \end{bmatrix}$$

$$\sim \begin{bmatrix} 1 & 3 & -1 \\ 0 & 2 & 6 \\ 0 & 6 & 3 \end{bmatrix}$$

$$\sim \begin{bmatrix} 1 & 3 & -1 \\ 0 & 2 & 6 \\ 0 & 0 & -15 \end{bmatrix} = U$$

Creation of L from I

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Add -2 times
row 1 to row 2.

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Add 1 times
row 1 to row 3.

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

Add -3 times
row 2 to row 3.

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 3 & 1 \end{bmatrix} \quad \blacksquare$$

We now illustrate how the record kept in L in Example 1 can be used to solve a linear system $Ax = b$ having the matrix A in Example 1 as coefficient matrix.

EXAMPLE 2 Use the record in L in Example 1 to solve the linear system $Ax = b$ given by

$$\begin{aligned} x_1 + 3x_2 - x_3 &= -4 \\ 2x_1 + 8x_2 + 4x_3 &= 2 \\ -x_1 + 3x_2 + 4x_3 &= 4. \end{aligned}$$

SOLUTION We use the record in L to find the column vector c that would occur if we were to reduce $[A \mid b]$ to $[U \mid c]$, using these same row operations:

Entry ℓ_{ij} in L **Meaning of the Entry** **Reduction of b to c**

$$\begin{array}{lll} \ell_{21} = 2 & \text{Add } -2 \text{ times row 1 to row 2.} & \begin{array}{l} b = \begin{bmatrix} -4 \\ 2 \\ 4 \end{bmatrix} \\ \sim \begin{bmatrix} -4 \\ 10 \\ 4 \end{bmatrix} \end{array} \\ \ell_{31} = -1 & \text{Add } -(-1) = 1 \text{ times row 1 to row 3.} & \begin{array}{l} \sim \begin{bmatrix} -4 \\ 10 \\ 0 \end{bmatrix} \end{array} \\ \ell_{32} = 3 & \text{Add } -3 \text{ times row 2 to row 3.} & \begin{array}{l} \sim \begin{bmatrix} -4 \\ 10 \\ -30 \end{bmatrix} = c. \end{array} \end{array}$$

If we put this result together with the matrix U obtained in Example 1, the reduced augmented matrix for the linear system becomes

$$[U | \mathbf{c}] = \left[\begin{array}{ccc|c} 1 & 3 & -1 & -4 \\ 0 & 2 & 6 & 10 \\ 0 & 0 & -15 & -30 \end{array} \right].$$

Back substitution then yields

$$\begin{aligned} x_3 &= \frac{-30}{-15} = 2, \\ x_2 &= \frac{10 - 6(2)}{2} = \frac{-2}{2} = -1, \\ x_1 &= -4 + 3 + 2 = 1. \end{aligned}$$

We give another example.

EXAMPLE 3 Let

$$A = \begin{bmatrix} 1 & -2 & 0 & 3 \\ -2 & 3 & 1 & -6 \\ -1 & 4 & -4 & 3 \\ 5 & -8 & 4 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 11 \\ -21 \\ -1 \\ 23 \end{bmatrix}.$$

Generate the matrix L while reducing the matrix A to U . Then use U and the record in L to solve $A\mathbf{x} = \mathbf{b}$.

SOLUTION We work in two columns again. This time we fix up a whole column of A in each step:

Reduction of A	Generation of L
$A = \begin{bmatrix} 1 & -2 & 0 & 3 \\ -2 & 3 & 1 & -6 \\ -1 & 4 & -4 & 3 \\ 5 & -8 & 4 & 0 \end{bmatrix}$	$I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
$\sim \begin{bmatrix} 1 & -2 & 0 & 3 \\ 0 & -1 & 1 & 0 \\ 0 & 2 & -4 & 6 \\ 0 & 2 & 4 & -15 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 5 & 0 & 0 & 1 \end{bmatrix}$
$\sim \begin{bmatrix} 1 & -2 & 0 & 3 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -2 & 6 \\ 0 & 0 & 6 & -15 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & -2 & 1 & 0 \\ 5 & -2 & 0 & 1 \end{bmatrix}$
$\sim \begin{bmatrix} 1 & -2 & 0 & 3 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -2 & 6 \\ 0 & 0 & 0 & 3 \end{bmatrix} = U$	$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & -2 & 1 & 0 \\ 5 & -2 & -3 & 1 \end{bmatrix}$

We now apply the record below the diagonal in L to the vector $\mathbf{b}$, working under the main diagonal down each column of the record in L in turn.

$$\text{First column of } L: \mathbf{b} = \begin{bmatrix} 11 \\ -21 \\ -1 \\ 23 \end{bmatrix} \sim \begin{bmatrix} 11 \\ 1 \\ -1 \\ 23 \end{bmatrix} \sim \begin{bmatrix} 11 \\ 1 \\ 10 \\ 23 \end{bmatrix} \sim \begin{bmatrix} 11 \\ 1 \\ 10 \\ -32 \end{bmatrix}.$$

$$\text{Second column of } L: \begin{bmatrix} 11 \\ 1 \\ 10 \\ -32 \end{bmatrix} \sim \begin{bmatrix} 11 \\ 1 \\ 12 \\ -32 \end{bmatrix} \sim \begin{bmatrix} 11 \\ 1 \\ 12 \\ -30 \end{bmatrix}.$$

$$\text{Third column of } L: \begin{bmatrix} 11 \\ 1 \\ 12 \\ -30 \end{bmatrix} \sim \begin{bmatrix} 11 \\ 1 \\ 12 \\ 6 \end{bmatrix}.$$

The augmented matrix

$$\left[\begin{array}{cccc|c} 1 & -2 & 0 & 3 & 11 \\ 0 & -1 & 1 & 0 & 1 \\ 0 & 0 & -2 & 6 & 12 \\ 0 & 0 & 0 & 3 & 6 \end{array} \right]$$

yields, upon back substitution,

$$x_4 = \frac{6}{3} = 2,$$

$$x_3 = \frac{12 - 12}{-2} = 0,$$

$$x_2 = \frac{1 - 0}{-1} = -1,$$

$$x_1 = 11 - 2 - 6 = 3.$$

Two questions come to mind:

1. Why bother to put the entries 1 down the main diagonal in L when they are not used?
2. If we add r times a row to another row while reducing A , why do we put $-r$ rather than r in the record in L , and then change back to r again when performing the operations on the column vector $\mathbf{b}$?

We do these two things only because the matrix L formed in this way has an interesting algebraic property, which we will discuss in a moment. In fact, when solving a large system using a computer, we certainly would not fuss with the entries 1 down the diagonal. Indeed, we can save memory space by not even generating a matrix L separate from the one being reduced. When creating a zero below the main diagonal, place the record $-r$ or r desired, as described in question 2 above, directly in the matrix being reduced at the

position where a zero is being created! The computer already has space reserved for an entry there. Just remember that the final matrix contains the desired entries of U on and above the diagonal and the record for L or $-L$ below the diagonal. *We will always use $-r$ rather than r as record entry in this text.* Thus, for the 4×4 matrix in Example 3, we obtain

$$\begin{bmatrix} 1 & -2 & 0 & 3 \\ -2 & -1 & 1 & 0 \\ -1 & -2 & -2 & 6 \\ 5 & -2 & -3 & 3 \end{bmatrix}, \quad \text{Combined } L \backslash U \text{ display} \quad (2)$$

where the black entries on or above the main diagonal give the essential data for U , and the color entries are the essential data for L .

Let us examine the efficiency of solving a system $Ax = b$ if U and L are already known. Each entry in the record in L requires one flop to execute when applying this record to reduce a column vector b . The number of entries is

$$1 + 2 + \cdots + n - 1 = \frac{(n-1)n}{2} = \frac{n^2}{2} - \frac{n}{2},$$

which is of order of magnitude $n^2/2$ for large n . We saw in Section 10.1 that back substitution requires about $n^2/2$ flops, too, giving a total of n^2 flops for large n to solve $Ax = b$, once U and L are known. If instead we computed A^{-1} and found $x = A^{-1}b$, the product $A^{-1}b$ would also require n^2 flops. But there are at least two advantages in using the LU -technique. First, finding U requires about $n^3/3$ flops for large n , whereas finding A^{-1} requires n^3 flops. (See Exercise 15 in Section 10.1.) If $n = 1000$, the difference in computer time is considerable. Second, more computer memory is used in reducing $[A \mid I]$ to $[I \mid A^{-1}]$ than is used in the efficient way we record L as we find U , illustrated in the combined $L \backslash U$ display (2).

We give a specific illustration in which keeping the record L is useful.

EXAMPLE 4 Let

$$A = \begin{bmatrix} 1 & 2 & -1 \\ -2 & -5 & 3 \\ -1 & -3 & 0 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 9 \\ -17 \\ -44 \end{bmatrix}.$$

Solve the linear system $A^3x = b$.

SOLUTION

We view $A^3x = b$ as $A(A^2x) = b$, and substitute $y = A^2x$ to obtain $Ay = b$. We can solve this equation for y . Then we write $A^2x = y$ as $A(Ax) = y$ or $Az = y$, where $z = Ax$. We then solve $Az = y$ for z . Finally, we solve $Ax = z$ for the desired x . Because we are using the same coefficient matrix A each time, it is efficient to find the matrices L and U and then to proceed as in Example 3 to find y , z , and x in turn.

We find that the matrices U and L are given by

$$U = \begin{bmatrix} 1 & 2 & -1 \\ 0 & -1 & 1 \\ 0 & 0 & -2 \end{bmatrix} \quad \text{and} \quad L = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 1 & 1 \end{bmatrix}.$$

Applying the record in L to $\mathbf{b}$, we obtain

$$\mathbf{b} = \begin{bmatrix} 9 \\ -17 \\ -44 \end{bmatrix} \sim \begin{bmatrix} 9 \\ 1 \\ -35 \end{bmatrix} \sim \begin{bmatrix} 9 \\ 1 \\ -36 \end{bmatrix}.$$

From the matrix U , we find that

$$\mathbf{y} = \begin{bmatrix} -7 \\ 17 \\ 18 \end{bmatrix}.$$

To solve $A\mathbf{z} = \mathbf{y}$, we apply the record in L to $\mathbf{y}$:

$$\mathbf{y} = \begin{bmatrix} -7 \\ 17 \\ 18 \end{bmatrix} \sim \begin{bmatrix} -7 \\ 3 \\ 11 \end{bmatrix} \sim \begin{bmatrix} -7 \\ 3 \\ 8 \end{bmatrix}.$$

Using U , we obtain

$$\mathbf{z} = \begin{bmatrix} 3 \\ -7 \\ -4 \end{bmatrix}.$$

Finally, to solve $A\mathbf{x} = \mathbf{z}$, we apply the record in L to $\mathbf{z}$:

$$\mathbf{z} = \begin{bmatrix} 3 \\ -7 \\ -4 \end{bmatrix} \sim \begin{bmatrix} 3 \\ -1 \\ -1 \end{bmatrix} \sim \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix}.$$

Using U , we find that

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

The routine LUFACOR in LINTEK can be used to find the matrices L and U ; it has an option for iteration to solve a system $A^m\mathbf{x} = \mathbf{b}$, as we did in Example 4.

The Factorization $A = LU$

This heading shows why the matrix L we described is algebraically interesting: we have $A = LU$.

EXAMPLE 5 Illustrate $A = LU$ for the matrices obtained in Example 3.

SOLUTION From Example 3, we have

$$LU = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & -2 & 1 & 0 \\ 5 & -2 & -3 & 1 \end{bmatrix} \begin{bmatrix} 1 & -2 & 0 & 3 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -2 & 6 \\ 0 & 0 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 1 & -2 & 0 & 3 \\ -2 & 3 & 1 & -6 \\ -1 & 4 & -4 & 3 \\ 5 & -8 & 4 & 0 \end{bmatrix} = A.$$

We have shown how to find a solution of $Ax = b$ from a factorization $A = LU$ by using the record in L to modify b to a vector c and then solving $Ux = c$ by back substitution. Approximately n^2 flops are required. An alternative method of determining x from $LUx = b$ is to view the equation as $L(Ux) = b$ letting $c = Ux$. We first solve $Lc = b$ for c by *forward substitution*, and then we solve $Ux = c$ by back substitution. Because each of the forward and back substitutions takes approximately $n^2/2$ flops, the total number required is again approximately n^2 flops. We illustrate this alternative method.

EXAMPLE 6 In Example 1, we found the factorization

$$\begin{array}{ccc} \begin{bmatrix} 1 & 3 & -1 \\ 2 & 8 & 4 \\ -1 & 3 & 4 \end{bmatrix} & = & \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 1 & 3 & -1 \\ 0 & 2 & 6 \\ 0 & 0 & -15 \end{bmatrix} \\ A & & L \quad U \end{array}$$

Use the method of forward substitution and back substitution to solve the

linear system $Ax = \begin{bmatrix} -4 \\ 2 \\ 4 \end{bmatrix}$, which we solved in Example 2.

SOLUTION First, we solve $Lc = b$ by forward substitution:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & -4 \\ 2 & 1 & 0 & 2 \\ -1 & 3 & 1 & 4 \end{array} \right]$$

$$c_1 = -4,$$

$$2c_1 + c_2 = 2, \quad c_2 = 2 + 8 = 10.$$

$$-c_1 + 3c_2 + c_3 = 4,$$

$$c_3 = c_1 - 3c_2 + 4 = -4 - 30 + 4 = -30.$$

Notice that this is the same c as was obtained in Example 2. The back substitution with $Ux = c$ of Example 2 then yields the same solution x . ■

Factorization of an invertible square matrix A into LU , with L lower triangular and U upper triangular, is not unique. For example, if r is a nonzero scalar, then rL is lower triangular and $(1/r)U$ is upper triangular, and if $A = LU$, then we also have $A = (rL)((1/r)U)$. But let D be the $n \times n$ diagonal matrix having the same main diagonal as U ; that is,

$$D = \begin{bmatrix} u_{11} & & 0 \\ & u_{22} & \\ 0 & & \ddots \\ & & & u_{nn} \end{bmatrix}.$$

Let U^* be the upper-triangular matrix obtained from U by multiplying the i th row by $1/u_{ii}$ for $i = 1, 2, \dots, n$. Then $U = DU^*$, and we have

$$A = LDU^*.$$

Now both L and U^* have all entries 1 on their main diagonals. This type of factorization is unique, as we now show.

THEOREM 10.1 Unique Factorization

Let A be an invertible $n \times n$ matrix. A factorization $A = LDU$, where

L is lower triangular with all main diagonal entries 1,

U is upper triangular with all main diagonal entries 1,

D is diagonal with all main diagonal entries nonzero,
is unique.

PROOF Suppose that $A = L_1 D_1 U_1$ and $A = L_2 D_2 U_2$ are two such factorizations. Observe that both L_1^{-1} and L_2^{-1} are also lower triangular, D_1^{-1} and D_2^{-1} are both diagonal, and U_1^{-1} and U_2^{-1} are both still upper triangular. Just think how the matrix reductions of $[L_1 | I]$ or $[D_1 | I]$ or $[U_1 | I]$ to find the inverses look. Furthermore, L_1^{-1} , L_2^{-1} , U_1^{-1} , and U_2^{-1} have all their main diagonal entries equal to 1.

Now from $L_1 D_1 U_1 = L_2 D_2 U_2$, we obtain

$$L_2^{-1} L_1 = D_2 U_2 U_1^{-1} D_1^{-1}. \quad (6)$$

We see that $L_2^{-1} L_1$ is again lower triangular with entries 1 on its main diagonal, whereas $D_2 U_2 U_1^{-1} D_1^{-1}$ is upper triangular. Equation (6) then shows that both matrices must be I , so $L_1 L_2^{-1} = I$ and $L_1 = L_2$. A similar argument starting over with $L_1 D_1 U_1 = L_2 D_2 U_2$ rewritten as

$$U_1 U_2^{-1} = D_1^{-1} L_1^{-1} L_2 D_2 \quad (7)$$

shows that $U_1 = U_2$. We then have $L_1 D_1 U_1 = L_1 D_2 U_1$, and multiplication on the left by L_1^{-1} and on the right by U_1^{-1} yields $D_1 = D_2$. ▲

Systems Requiring Row Interchanges

Let A be an invertible square matrix whose row reduction to an upper-triangular matrix U requires at least one row interchange. Then not all elementary matrices corresponding to the necessary row operations add multiples of row vectors to other row vectors. We can still write

$$E_h E_{h-1} \cdots E_2 E_1 A = U,$$

so

$$A = E_1^{-1} E_2^{-1} \cdots E_{h-1}^{-1} E_h^{-1} U,$$

but now that some of these E_i^{-1} interchange rows, their product may not be lower triangular. However, after we discover which row interchanges are necessary, we could start over, and make these necessary row interchanges in the matrix A before we start creating zeros below the diagonal. We can see that the upper-triangular matrix U obtained would still be the same. Suppose, for example, that to obtain a nonzero element in pivot position in the i th row we interchange this i th row with a k th row farther down in the matrix. The new i th row will be the same as though it had been put in the i th row position before the start of the reduction; in either case, it has been modified during the reduction only by the addition of multiples of rows *above* the i th row position. As multiples of it are now added to rows below the i th row position, the same rows (except possibly for order) below the i th row position are created whether row interchange is performed during reduction or is completed before reduction starts.

Interchanging some rows before the start of the reduction amounts to multiplying A on the left by a sequence of elementary row-interchange matrices. Any product of elementary row-interchange matrices is called a **permutation matrix**. Thus we can form PA for a permutation matrix P , and PA will then admit a factorization $PA = LU$. We state this as a theorem.

THEOREM 10.2 LU-Factorization

Let A be an invertible square matrix. Then there exists a permutation matrix P , a lower-triangular matrix L , and an upper-triangular matrix U such that $PA = LU$.

EXAMPLE 7 Illustrate Theorem 10.2 for the matrix

$$A = \begin{bmatrix} 1 & 3 & 2 \\ -2 & -6 & 1 \\ 2 & 5 & 7 \end{bmatrix}.$$

SOLUTION Starting to reduce A to upper-triangular form, we have

$$\begin{bmatrix} 1 & 3 & 2 \\ -2 & -6 & 1 \\ 2 & 5 & 7 \end{bmatrix} \sim \begin{bmatrix} 1 & 3 & 2 \\ 0 & 0 & 5 \\ 0 & -1 & 3 \end{bmatrix},$$

and we now find it necessary to interchange rows 2 and 3, which will then produce the desired U . Thus we take

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix},$$

and we have

$$PA = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 5 & 7 \\ -2 & -6 & 1 \end{bmatrix} \sim \begin{bmatrix} 1 & 3 & 2 \\ 0 & -1 & 3 \\ 0 & 0 & 5 \end{bmatrix} = U.$$

The record matrix L for reduction of PA becomes

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix},$$

and we confirm that

$$PA = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 5 & 7 \\ -2 & -6 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 3 & 2 \\ 0 & -1 & 3 \\ 0 & 0 & 5 \end{bmatrix} = LU. \quad \blacksquare$$

Suppose that, when solving a large square linear system $Ax = b$ with a computer, keeping the record L , we find that row interchanges are advisable. (See the discussion of partial pivoting in Section 10.3.) We could keep going and make the row interchanges to find an upper-triangular matrix; we could then start over, make those row interchanges first, just as in Example 7, to obtain a matrix PA , and then obtain U and L for the matrix PA instead. Of course, we would first have to compute Pb when solving $Ax = b$, because the system would then become $PAx = Pb$, before we could proceed with the record L and back substitution. This is an undesirable procedure, because it requires reduction of both A and PA , taking a total of about $2n^3/3$ flops, assuming that A is $n \times n$ and n is large. Surely it would be better to devise a method of record-keeping that would also keep track of any row interchanges as they occurred, and would then apply this improved record to b and use back substitution to find the solution. We toss this suggestion out for enthusiastic programmers to consider on their own.

SUMMARY

1. If A is an $n \times n$ invertible matrix that can be row-reduced to an upper-triangular matrix U without row interchanges, there exists a lower-triangular $n \times n$ matrix L such that $A = LU$.
2. The matrix L in summary item 1 can be found as follows. Start with the $n \times n$ identity matrix I . If during the reduction of A to U , r times row i is added to row k , replace the zero in row k and column i of the identity matrix by $-r$. The final result obtained from the identity matrix is the matrix L .
3. Once A has been reduced to U and L has been found, a computer can find the solution of $Ax = b$ for a new column vector b , using about n^2 flops for large n .

4. If A is as described in summary item 1, then A has a unique factorization of the form $A = LDU$, where

L is lower triangular with all diagonal entries 1,

U is upper triangular with all diagonal entries 1, and

D is a diagonal matrix with all diagonal entries nonzero.

5. For any invertible matrix A , there exists a permutation matrix P such that PA can be row-reduced to an upper-triangular matrix U and has the properties described for A in summary items 1, 2, 3, and 4.

EXERCISES

1. Discuss briefly the need to worry about the time required to create the record matrix L when solving

$$Ax = b_1, b_2, \dots, b_r$$

for a large $n \times n$ matrix A .

2. Is there any practical value in creating the record matrix L when one needs to solve only a single square linear system $Ax = b$, as we did in Example 3?

In Exercises 3–7, find the solution of $Ax = b$ from the given combined $L \setminus U$ display of the matrix A and the given vector b .

3. $L \setminus U = \begin{bmatrix} 1 & 4 \\ 3 & -2 \end{bmatrix}, b = \begin{bmatrix} 2 \\ -4 \end{bmatrix}$

4. $L \setminus U = \begin{bmatrix} -2 & 5 \\ -2 & 1 \end{bmatrix}, b = \begin{bmatrix} 3 \\ -7 \end{bmatrix}$

5. $L \setminus U = \begin{bmatrix} 1 & -3 & 4 \\ -2 & -1 & 9 \\ 0 & 1 & -6 \end{bmatrix}, b = \begin{bmatrix} 2 \\ -2 \\ 8 \end{bmatrix}$

6. $L \setminus U = \begin{bmatrix} 1 & -4 & 2 \\ 0 & 2 & -1 \\ 3 & 2 & -2 \end{bmatrix}, b = \begin{bmatrix} -3 \\ 2 \\ 3 \end{bmatrix}$

7. $L \setminus U = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & -1 & 2 & 1 \\ -1 & -2 & 1 & 3 \\ 2 & 1 & -1 & 3 \end{bmatrix}, b = \begin{bmatrix} 4 \\ 7 \\ -8 \\ 14 \end{bmatrix}$

In Exercises 8–13, let A be the given matrix. Find a permutation matrix P , if necessary, and matrices L and U such that $PA = LU$. Check the

answer, using matrix multiplication. Then solve the system $Ax = b$, using P , L , and U .

8. $A = \begin{bmatrix} 2 & -1 \\ 6 & -5 \end{bmatrix}, b = \begin{bmatrix} 8 \\ 32 \end{bmatrix}$

9. $A = \begin{bmatrix} 2 & 1 & -3 \\ 6 & 3 & -8 \\ 2 & -1 & 5 \end{bmatrix}, b = \begin{bmatrix} 6 \\ 17 \\ 0 \end{bmatrix}$

10. $A = \begin{bmatrix} 1 & 1 & -3 \\ 0 & 1 & 1 \\ 3 & -1 & 1 \end{bmatrix}, b = \begin{bmatrix} -13 \\ 6 \\ -7 \end{bmatrix}$

11. $A = \begin{bmatrix} 1 & 2 & -1 \\ 3 & 7 & 2 \\ 4 & -2 & 1 \end{bmatrix}, b = \begin{bmatrix} -3 \\ 1 \\ -2 \end{bmatrix}$

12. $A = \begin{bmatrix} 1 & -4 & 1 & -2 \\ 0 & 2 & -1 & 1 \\ 2 & -7 & -2 & 1 \\ 0 & 3 & 0 & -4 \end{bmatrix}, b = \begin{bmatrix} -9 \\ 6 \\ 10 \\ -16 \end{bmatrix}$

13. $A = \begin{bmatrix} 1 & 2 & -3 & 0 \\ 2 & 5 & -6 & 0 \\ -1 & -2 & 1 & -1 \\ 4 & 10 & -9 & 1 \end{bmatrix}, b = \begin{bmatrix} 8 \\ 17 \\ -8 \\ 33 \end{bmatrix}$

In Exercises 14–17, proceed as in Example 4 to solve the given system.

14. Solve $A^2x = b$ if $A = \begin{bmatrix} 1 & 1 \\ 3 & 0 \end{bmatrix}, b = \begin{bmatrix} -11 \\ -6 \end{bmatrix}$.

15. Solve $A^4x = b$ if $A = \begin{bmatrix} -1 & 2 \\ 2 & -3 \end{bmatrix},$

$$b = \begin{bmatrix} 144 \\ -233 \end{bmatrix}.$$

Solve $A^2x = b$ if $A = \begin{bmatrix} 2 & -1 & 0 \\ 4 & -1 & 2 \\ -6 & 2 & 0 \end{bmatrix}$,

$$b = \begin{bmatrix} -2 \\ 14 \\ 12 \end{bmatrix}.$$

Solve $A^3x = b$ if $A = \begin{bmatrix} -1 & 0 & 1 \\ 3 & 1 & 0 \\ -2 & 0 & 4 \end{bmatrix}$,

$$b = \begin{bmatrix} 27 \\ 29 \\ 122 \end{bmatrix}.$$

Exercises 18–20, find the unique factorization LU for the given matrix A .

The matrix in Exercise 8

The matrix in Exercise 10

The matrix in Exercise 11

Mark each of the following True or False.

- a. Every matrix A has an LU -factorization.
- b. Every square matrix A has an LU -factorization.
- c. Every square matrix A has a factorization $P^{-1}LU$ for some permutation matrix P .
- d. If an LU -factorization of an $n \times n$ matrix A is known, using this factorization to solve a large linear system with coefficient matrix A requires about $n^2/2$ flops.
- e. If an LU -factorization of an $n \times n$ matrix A is known, using this factorization to solve a large linear system with coefficient matrix A requires about n^2 flops.
- f. If $A = LU$, then this is the only factorization of this type for A .
- g. If $A = LU$, then the matrix L can be regarded as a record of the steps in the row reduction of A to U .
- h. All three types of elementary row operations used to reduce A to U can be recorded in the matrix L in an LU -factorization of A .
- i. Two of the three types of elementary row operations used to reduce A to U can be recorded in the matrix L in an LU -factorization of A .
- j. One can solve a linear system $LUx = b$ by means of a forward substitution followed by a back substitution.

 In Exercises 22–24, use the routine **LUFAC** in **LINTEK** to find the combined LU display (2) for the matrix A . Then solve $Ax = b_i$ for each of the column vectors b_i .

$$22. A = \begin{bmatrix} 2 & 1 & -3 & 4 & 0 \\ 1 & 4 & 6 & -1 & 5 \\ -2 & 0 & 3 & 1 & 4 \\ 6 & 1 & 2 & 8 & -3 \\ 4 & 1 & 3 & 2 & 1 \end{bmatrix};$$

$$b_i = \begin{bmatrix} 13 \\ 7 \\ 4 \\ 27 \\ 17 \end{bmatrix}, \begin{bmatrix} 10 \\ 0 \\ 10 \\ 60 \\ 20 \end{bmatrix}, \text{ and } \begin{bmatrix} -20 \\ 96 \\ 53 \\ 5 \\ 26 \end{bmatrix} \text{ for } i = 1, 2, \text{ and } 3, \text{ respectively.}$$

$$23. A = \begin{bmatrix} 1 & 3 & -5 & 2 & 1 \\ 4 & -6 & 10 & 8 & 3 \\ 3 & 6 & -1 & 4 & 7 \\ 2 & 1 & 11 & -3 & 13 \\ -6 & 3 & 1 & 4 & 21 \end{bmatrix};$$

$$b_i = \begin{bmatrix} 0 \\ 20 \\ -9 \\ -31 \\ -43 \end{bmatrix}, \begin{bmatrix} -48 \\ 218 \\ 0 \\ 100 \\ 143 \end{bmatrix}, \text{ and } \begin{bmatrix} 87 \\ -151 \\ 102 \\ -46 \\ 223 \end{bmatrix}$$

for $i = 1, 2, \text{ and } 3$, respectively.

$$24. A = \begin{bmatrix} -1 & 3 & 2 & 3 & 4 & 6 \\ 3 & 1 & 0 & -2 & 1 & 7 \\ 6 & 8 & -2 & 1 & 4 & 6 \\ 4 & -2 & 3 & 1 & 7 & 6 \\ 3 & -2 & 1 & 10 & 8 & 0 \\ 4 & -5 & 8 & -3 & 2 & 10 \end{bmatrix};$$

$$b_i = \begin{bmatrix} 19 \\ 1 \\ -8 \\ 82 \\ 76 \\ 103 \end{bmatrix}, \begin{bmatrix} 82 \\ 2 \\ 82 \\ 31 \\ 92 \\ -61 \end{bmatrix}, \text{ and } \begin{bmatrix} 25 \\ -3 \\ -24 \\ -73 \\ -115 \\ 20 \end{bmatrix}$$

for $i = 1, 2, \text{ and } 3$, respectively.

In Exercises 25–29, use the routine **LUFAC** in **LINTEK** to solve the indicated system.

25. $A^5x = b$ for A and b in Exercise 14

26. $A^7x = b$ for A and b in Exercise 15

27. $A^6x = b$ for A and b in Exercise 16

28. $A^8x = b$ for A and b in Exercise 17
 29. $A^3x = b_1$ for A and b_1 in Exercise 24

The routine *TIMING* in *LINTEK* has an option to time the formation of the combined $L\backslash U$ display from a matrix A . The user specifies the size n for an $n \times n$ linear system $Ax = b$. The program then generates an $n \times n$ matrix A with entries in the interval $[-20, 20]$ and creates the $L\backslash U$ display shown in matrix (2) in the text. The time required for the reduction is indicated. The user may then specify a number s for solution of $Ax = b_1, b_2, \dots, b_s$. The computer generates a column vector and solves s systems, using the record in L and back substitution; the time to find these s solutions is also indicated. Recall that the reduction of A should require about $n^3/3$ flops for

large n , and the solution for each column vector should require about n^2 flops. In Exercises 30–34 use the routine *TIMING* and see if the ratios of times obtained seem to conform to the ratios of the numbers of flops required, at least as n increases. Compare the time required to solve one system, including the computation of $L\backslash U$, with the time required to solve a system of the same size using the Gauss method with back substitution. The times should be about the same

30. $n = 4; s = 1, 3, 12$
 31. $n = 10; s = 1, 5, 10$
 32. $n = 16; s = 1, 8, 16$
 33. $n = 24; s = 1, 12, 24$
 34. $n = 30; s = 1, 15, 30$

MATLAB

MATLAB uses LU -factorization, for example, in its computation of $A\backslash B$. (Recall that $A\backslash B = A^{-1}B$ if A is a square matrix.) In finding an LU -factorization of A , MATLAB uses partial pivoting (row swapping to make pivots as large as possible) for increased accuracy (see Section 10.3). It obtains a lower-triangular matrix L and an upper-triangular matrix U such that $LU = PA$, where P is a permutation matrix as illustrated in Example 7. The MATLAB command

$$[L,U,P] = \text{lu}(A)$$

produces these matrices L , U , and P . In pencil-and-paper work, we like small pivots rather than large ones, and we swapped rows in Exercises 8–13 only if we encountered a zero pivot. Thus this MATLAB command cannot be used to check our answers to Exercises 8–13.

10.3

PIVOTING, SCALING, AND ILL-CONDITIONED MATRICES

Some Problems Encountered with Computers

A computer can't do absolutely precise arithmetic with real numbers. For example, any computer can only compute using an approximation of the number $\sqrt{2}$, never with the number $\sqrt{2}$ itself. Computers work using base-2 arithmetic, representing numbers as strings of zeros and ones. But they encounter the same problems that we encounter if we pretend that we are a computer using base-10 arithmetic and are capable of handling only some fixed finite number of significant digits. We will illustrate computer problems using base-10 notation, because it is more familiar to all of us.

Suppose that our base-10 computer is asked to compute the quotient $\frac{1}{3}$, and suppose that it can represent a number in floating-point arithmetic using eight significant figures. It represents $\frac{1}{3}$ in **truncated** form as 0.33333333. It may represent $\frac{2}{3}$ as 0.66666667, **rounding off** to eight significant figures. For convenience, we will refer to all errors generated by either truncation or roundoff as **roundoff errors**.

Most people realize that in an extended arithmetic computation by a computer, the roundoff error can accumulate to such an extent that the final result of the computation becomes meaningless. We will see that there are at least two situations in which a computer cannot meaningfully execute even a single arithmetic operation. In order to avoid working with long strings of digits, we assume that our computer can compute only to three significant figures in its work. Thus, our three-figure, base-10 computer computes the quotient $\frac{2}{3}$ as 0.666. We box the first computer problem that concerns us, and give an example.

Addition of two numbers of very different magnitude may result in the loss of some or even all of the significant figures of the smaller number.

EXAMPLE 1 Evaluate our three-figure computer's computation of

$$45.1 + .0725.$$

SOLUTION Because it can handle only three significant figures, our computer represents the actual sum 45.1725 as 45.1. In other words, the second summand .0725 might as well be zero, so far as our computer is concerned. The datum .0725 is completely lost. ■

The difficulty illustrated in Example 1 can cause serious problems in attempting to solve a linear system $Ax = b$ with a computer, as we will illustrate in a moment. First we box another problem a computer may have.

Subtraction of nearly equal numbers can result in a loss of significant figures.

EXAMPLE 2 Evaluate our three-figure computer's computation of

$$\frac{2}{3} - \frac{665}{1000}.$$

SOLUTION The actual difference is

$$.666666 \dots - .665 = .00166666 \dots$$

However, our three-figure computer obtains

$$.666 - .665 = .001.$$

Two of the three significant figures with which the computer can work have been lost. ■

The difficulty illustrated in Example 2 is encountered if one tries to use a computer to do differential calculus.

The cure for both difficulties is to have the computer work with more significant figures. But using more significant figures requires the computer to take more time to execute a program, and of course the same errors can occur "farther out." For example, the typical microcomputer of this decade, with software designed for routine computations, will compute $10^{10} + 10^{-10}$ as 10^{10} , losing the second summand 10^{-10} entirely.

Partial Pivoting

In row reduction of a matrix to echelon form, a technique called *partial pivoting* is often used. In **partial pivoting**, one interchanges the row in which the pivot is to occur with a row farther down, if necessary, so that the pivot becomes as large in absolute value as possible. To illustrate, suppose that row reduction of a matrix to echelon form leads to an intermediate matrix

$$\begin{bmatrix} 2 & 8 & -1 & 3 & 4 \\ 0 & -2 & 3 & -5 & 6 \\ 0 & 4 & 1 & 2 & 0 \\ 0 & -7 & 3 & 1 & 4 \end{bmatrix}.$$

Using partial pivoting, we would then interchange rows 2 and 4 to use the entry -7 of maximum magnitude among the possibilities -2 , 4 , and -7 for pivots in the second column. That is, we would form the matrix

$$\begin{bmatrix} 2 & 8 & -1 & 3 & 4 \\ 0 & -7 & 3 & 1 & 4 \\ 0 & 4 & 1 & 2 & 0 \\ 0 & -2 & 3 & -5 & 6 \end{bmatrix}$$

and continue with the reduction of this matrix.

We show by example the advantage of partial pivoting. Let us consider the linear system

$$\begin{aligned} .01x_1 + 100x_2 &= 100 \\ -100x_1 + 200x_2 &= 100. \end{aligned} \quad (1)$$

EXAMPLE 3 Find the actual solution of linear system (1). Then compare the result with that obtained by a three-figure computer using the Gauss method with back substitution, but without partial pivoting.

SOLUTION First we find the actual solution:

$$\left[\begin{array}{cc|c} .01 & 100 & 100 \\ -100 & 200 & 100 \end{array} \right] \sim \left[\begin{array}{cc|c} .01 & 100 & 100 \\ 0 & 1,000,200 & 1,000,100 \end{array} \right],$$

and back substitution yields

$$x_2 = \frac{1,000,100}{1,000,200},$$

$$x_1 = \left(100 - \frac{100,010,000}{1,000,200} \right) 100 = \frac{1,000,000}{1,000,200}.$$

Thus, $x_1 \approx .9998$ and $x_2 \approx .9999$. On the other hand, our three-figure computer obtains

$$\left[\begin{array}{cc|c} .01 & 100 & 100 \\ -100 & 200 & 100 \end{array} \right] \sim \left[\begin{array}{cc|c} .01 & 100 & 100 \\ 0 & 1,000,000 & 1,000,000 \end{array} \right],$$

which leads to

$$x_2 = 1,$$

$$x_1 = (100 - 100)100 = 0.$$

The x_1 -parts of the solution are very different. Our three-figure computer completely lost the second-row data entries 200 and 100 in the matrix when it added 10,000 times the first row to the second row. ■

EXAMPLE 4 Find the solution to system (1) that our three-figure computer would obtain using partial pivoting.

SOLUTION Using partial pivoting, the three-figure computer obtains

$$\left[\begin{array}{cc|c} .01 & 100 & 100 \\ -100 & 200 & 100 \end{array} \right] \sim \left[\begin{array}{cc|c} -100 & 200 & 100 \\ .01 & 100 & 100 \end{array} \right]$$

$$\sim \left[\begin{array}{cc|c} -100 & 200 & 100 \\ 0 & 100 & 100 \end{array} \right],$$

which yields

$$x_2 = 1,$$

$$x_1 = \frac{100 - 200}{-100} = 1.$$

This is close to the solution $x_1 \approx .9998$ and $x_2 \approx .9999$ obtained in Example 3, and it is much better than the erroneous $x_1 = 0$, $x_2 = 1$ that the three-figure computer obtained in Example 3 without pivoting. The data entries 200 and 100 in the second row of the initial matrix were not lost in this computation. ■

To understand completely the reason for the difference between the solutions in Example 3 and in Example 4, consider again the linear system

$$\begin{aligned} .01x_1 + 100x_2 &= 100 \\ -100x_1 + 200x_2 &= 100, \end{aligned}$$

which was solved in Example 3. Multiplication of the first row by 10,000 produced a coefficient of x_2 of such magnitude compared with the second equation coefficient 200 that the coefficient 200 was totally destroyed in the ensuing addition. (Remember that we are using our three-figure computer. In a less dramatic example, some significant figures of the smaller coefficient might still contribute to the sum.) If we use partial pivoting, the linear system becomes

$$\begin{aligned} -100x_1 + 200x_2 &= 100 \\ .01x_1 + 100x_2 &= 100 \end{aligned}$$

and multiplication of the first equation by a number less than 1 does not threaten the significant digits of the numbers in the second equation. This explains the success of partial pivoting in this situation.

Suppose now that we multiply the first equation of system (1) by 10,000, which of course does not alter the solution of the system. We then obtain the linear system

$$\begin{aligned} 100x_1 + 1,000,000x_2 &= 1,000,000 \\ -100x_1 + 200x_2 &= 100. \end{aligned} \quad (2)$$

If we solved system (2) using *partial pivoting*, we would not interchange the rows, because -100 is of no greater magnitude than 100 . Addition of the first row to the second by our three-figure computer again totally destroys the coefficient 200 in the second row. Exercise 1 shows that the erroneous solution $x_1 = 0$, $x_2 = 1$ is again obtained. We could avoid this problem by *full pivoting*. In full pivoting, columns are also interchanged, if necessary, to make pivots as large as possible. That is, if a pivot is to be found in the row i and column j position of an intermediate matrix G , then not only rows but also columns are interchanged as needed to put the entry g_{ij} of greatest magnitude, where $r \geq i$ and $s \geq j$, in the pivot position. Thus, full pivoting for system (2) will lead to the matrix

$$\begin{array}{cc|c} & x_2 & x_1 \\ \hline 1,000,000 & 100 & 1,000,000 \\ 200 & -100 & 100 \end{array} \quad (3)$$

Exercise 2 illustrates that row reduction of matrix (3) by our three-figure computer gives a reasonable solution of system (2). Notice, however, that we now have to do some bookkeeping and must remember that the entries in column 1 of matrix (3) are really the coefficients of x_2 , not of x_1 . For a matrix of any size, the search for elements of maximum magnitude and the bookkeeping required in full pivoting take a lot of computer time. Partial pivoting

frequently used, representing a compromise between time and accuracy. The routine MATCOMP in LINTEK uses partial pivoting in its Gauss-Jordan reduction to reduced echelon form. Thus, if system (1) is modified so that the Gauss-Jordan method without pivoting fails to give a reasonable solution for a 20-figure computer, MATCOMP could probably handle it. However, one can no doubt create a similar modification of the 2×2 system (2) for which MATCOMP would give an erroneous solution.

Scaling

We display again system (2):

$$\begin{array}{rcl} 100x_1 + 1,000,000x_2 & = & 1,000,000 \\ -100x_1 + 200x_2 & = & 100. \end{array}$$

We might recognize that the number 1,000,000 dangerously dominates the data entries in the second row, at least as far as our three-figure computer is concerned. We might multiply the first equation in system (2) by .0001 to cut those two numbers down to size, essentially coming back to system (1). Partial pivoting handles system (1) well. Multiplication of an equation by a nonzero constant for such a purpose is known as **scaling**. Of course, one could equivalently scale by multiplying the second equation in system (2) by 10,000, to bring its coefficients into line with the large numbers in the first equation.

We box one other computer problem that can sometimes be addressed by scaling. In reducing a matrix to echelon form, we need to know whether the entry that appears in a pivot position as we start work on a column is truly nonzero, and indeed, whether there is any truly nonzero entry from that column to serve as pivot.

Due to roundoff error, a computed number that should be zero is quite likely to be of small, nonzero magnitude.

Taking this into account, one usually programs row-echelon reduction so that entries of unexpectedly small size are changed to zero. MATCOMP finds the smallest nonzero magnitude m among 1 and all the coefficient data supplied for the linear system, and sets $E = rm$, where r is specified by the user. (Default r is .0001.) In reduction of the coefficient matrix, a computed entry of magnitude less than E is replaced by zero. The same procedure is followed in YUREDUCE. Whatever computed number we program a computer to choose for E in a program such as MATCOMP, we will be able to devise some linear system for which E is either too large or too small to give the correct result.

A procedure equivalent to the one in MATCOMP that we just outlined is to *scale* the original data for the linear system in such a way that the smallest nonzero entry is of magnitude roughly 1, and then always use the same value, perhaps 10^{-4} , for E .

When one of the authors first started experimenting with a computer, was horrified to discover that a matrix inversion routine built into a mainframe BASIC program of a major computer company would refuse to invert a matrix if all the entries were small enough. An error message such

“Nearly singular matrix. Inversion impossible.”

would appear. He considers a 2×2 matrix

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

to be nearly singular if and only if lines in the plane having equations of the form

$$ax + by = r$$

$$cx + dy = s$$

are nearly parallel. Now the lines

$$10^{-8}x + 0y = 1$$

$$0x + 10^{-9}y = 1$$

are actually perpendicular, the first one being vertical and the second horizontal. This is as far away from parallel as one can get! It annoyed the author greatly to have the matrix

$$\begin{bmatrix} 10^{-8} & 0 \\ 0 & 10^{-9} \end{bmatrix}$$

called “nearly singular.” The inverse is obviously

$$\begin{bmatrix} 10^8 & 0 \\ 0 & 10^9 \end{bmatrix}$$

A scaling routine was promptly written to be executed before calling the inversion routine. The matrix was multiplied by a constant that would bring the smallest nonzero magnitude to at least 1, and then the inversion subroutine was used, and the result rescaled to provide the inverse of the original matrix. For example, applied to the matrix just discussed, this procedure becomes

$$\begin{aligned} \begin{bmatrix} 10^{-8} & 0 \\ 0 & 10^{-9} \end{bmatrix} & \text{multiplied by } 10^9 \text{ becomes } \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}, \\ & \text{inverted becomes } \begin{bmatrix} \frac{1}{10} & 0 \\ 0 & 1 \end{bmatrix}, \\ & \text{multiplied by } 10^9 \text{ becomes } \begin{bmatrix} 10^8 & 0 \\ 0 & 10^9 \end{bmatrix} \end{aligned}$$

Having more programming experience now, this author is much more charitable and understanding. The user may also find unsatisfactory things about MATCOMP. We hope that this little anecdote has helped explain the notion of scaling.

Ill-Conditioned Matrices

The line $x + y = 100$ in the plane has x -intercept 100 and y -intercept 100, as shown in Figure 10.1. The line $x + .9y = 100$ also has x -intercept 100, but it has y -intercept larger than 100. The two lines are almost parallel. The common x -intercept shows that the solution of the linear system

$$\begin{aligned} x + y &= 100 \\ x + .9y &= 100 \end{aligned} \quad (4)$$

is $x = 100, y = 0$, as illustrated in Figure 10.2. Now the line $.9x + y = 100$ has y -intercept 100 but x -intercept larger than 100, so the linear system

$$\begin{aligned} x + y &= 100 \\ .9x + y &= 100 \end{aligned} \quad (5)$$

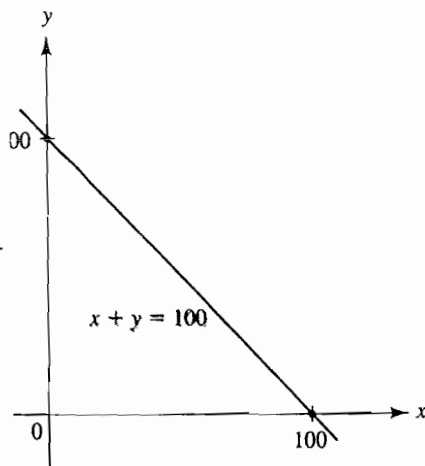


FIGURE 10.1
The line $x + y = 100$.

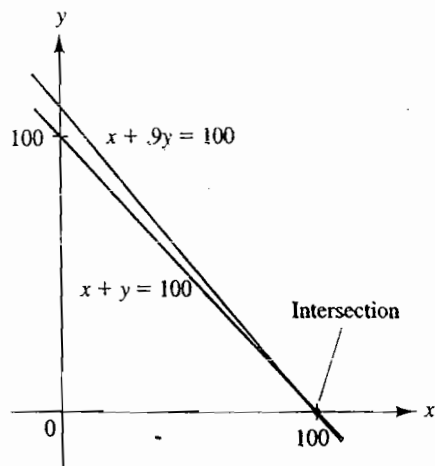


FIGURE 10.2
The lines are almost parallel.

has the very different solution $x = 0, y = 100$, as shown in Figure 10.3. Systems (4) and (5) are examples of **ill-conditioned** or **unstable** systems: small changes in the coefficients or in the constants on the right-hand sides can produce very great changes in the solutions. We say that a matrix A is **ill-conditioned** if the linear system $Ax = b$ having A as coefficient matrix is ill-conditioned. For two equations in two unknowns, solving an ill-conditioned system corresponds to finding the intersection of two nearly parallel lines, as shown in Figure 10.4. Changing a coefficient of x or y slightly in one equation changes the slope of that line only slightly, but it may generate a big change in the location of the point of intersection of the two lines.

Computers have a lot of trouble finding accurate solutions of ill-conditioned systems such as systems (4) and (5), because the small round-off errors created by the computer can produce large changes in the solutions. Pivoting and scaling usually don't help the situation; the systems are basically unstable. Notice that the coefficients of x and y in systems (4) and (5) are of comparable magnitude.

Among the most famous ill-conditioned matrices are the Hilbert matrices. These are very bad matrices named after a very good mathematician, David Hilbert! (See the historical note on page 444.) The entry in the i th row and j th column of a Hilbert matrix is $1/(i + j - 1)$. Thus, if we let H_n be the $n \times n$ Hilbert matrix, we have

$$H_2 = \begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{3} \end{bmatrix}, \quad H_3 = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}, \quad \text{and so on.}$$

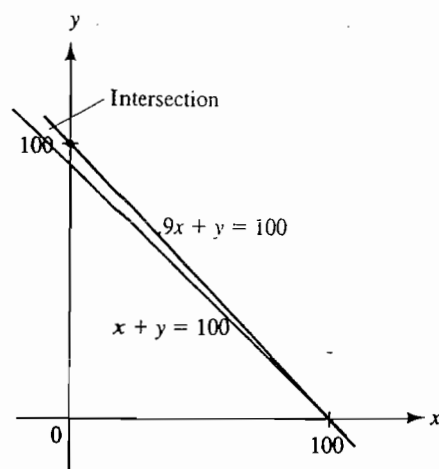


FIGURE 10.3

A very different solution from the one in Figure 10.2.

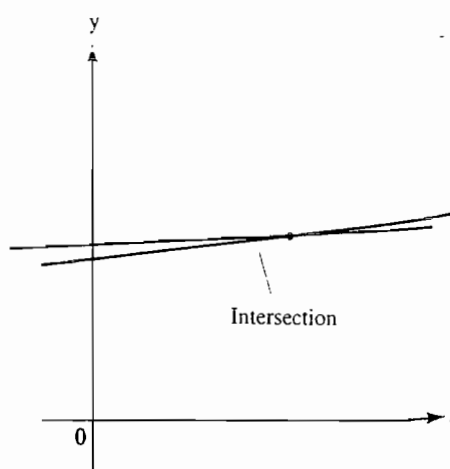


FIGURE 10.4

The intersection of two nearly parallel lines.

It can be shown that H_n is invertible for all n , so a square linear system $H_n \mathbf{x} = \mathbf{b}$ has a unique solution, but the solution may be very hard to find. When the matrix is reduced to echelon form, entries of surprisingly small magnitude appear. Scaling of a row in which all entries are close to zero may help a bit.

Bad as the Hilbert matrices are, powers of them are even worse. The software we make available includes a routine called HILBERT, which is modeled on YUREDUCE. The computer generates a Hilbert matrix of the size we specify, up to 10×10 . It will then raise the matrix to the power 2, 4, 8, or 16 if we so request. We may then proceed roughly as in YUREDUCE. Routines such as YUREDUCE and HILBERT should help us understand this section, because we can watch and see just what is happening as we reduce a matrix. MATCOMP, which simply spits out answers, may produce an absurd result, but we have no way of knowing exactly where things went wrong.

SUMMARY

1. Addition of numbers of very different magnitudes by a computer can cause loss of some or all of the significant figures in the number of smaller magnitude.
2. Subtraction of nearly equal numbers by a computer can cause loss of significant figures.
3. Due to roundoff error, a computer may obtain a nonzero value for a number that should be zero. To attempt to handle this problem, a computer program might assign the value zero to certain computed numbers whenever the numbers have a magnitude less than some predetermined small positive number.
4. In partial pivoting, the pivot in each column is created by swapping rows, if necessary, so that the pivot has at least the maximum magnitude of any entry below it in that column. Partial pivoting may be helpful in avoiding the problem stated in summary item 1.
5. In full pivoting, columns are interchanged as well as rows, if necessary, to create pivots of maximum possible magnitude. Full pivoting requires much more computer time than does partial pivoting, and bookkeeping is necessary to keep track of the relationship between the columns and the variables. Partial pivoting is more commonly used.
6. *Scaling*, which is multiplication of a row by a nonzero constant, can be used to reduce the sizes of entries that threaten to dominate entries in lower rows, or to increase the sizes of entries in a row where some entries are very small.
7. A linear system $A\mathbf{x} = \mathbf{b}$ is ill-conditioned or unstable if small changes in the numbers can produce large changes in the solution. The matrix A is then also called *ill-conditioned*.

8. Hilbert matrices, which are square matrices with entry $1/(i + j - 1)$ in the i th row and j th column, are examples of ill-conditioned matrices.
9. With present technology, it appears hopeless to write a computer program that will successfully handle every linear system involving even very small coefficient matrices—say, of size at most 10×10 .

EXERCISES

1. Find the solution by a three-figure computer of the system

$$\begin{aligned} 100x_1 + 1,000,000x_2 &= 1,000,000 \\ -100x_1 + 200x_2 &= 100 \end{aligned}$$

using just partial pivoting.

2. Repeat Exercise 1, but use full pivoting.
3. Find the solution, without pivoting, by a five-figure computer of the linear system in Exercise 1. Is the solution reasonably accurate? If so, modify the system a bit to obtain one for which a five-figure computer does not find a reasonable solution without pivoting.
4. Modify the linear system in Exercise 1 so that an eight-figure computer (roughly the usual single-precision computer) will not obtain a reasonable solution without partial pivoting.
5. Repeat Exercise 4 for an 18-figure computer (roughly the usual double-precision computer).
6. Find a linear system with two equations in x and y such that a change of .001 in the coefficient of x in the second equation produces a change of at least 1,000,000 in both of the values x and y in a solution.
7. Let A be an invertible square matrix. Show that the following scaling routine for finding A^{-1} using a computer is *mathematically* correct.
 - a. Find the minimum absolute value m of all nonzero entries in A .
 - b. Multiply A by an integer $n > 1/m$.
 - c. Find the inverse of the resulting matrix.
 - d. Multiply the matrix obtained in part c by n to get A^{-1} .



Use the routines *MATCOMP*, *YUREDUCE*, and *HILBERT* in *LINTEK* for Exercises 8–13.

8. Use *YUREDUCE* to solve the system

$$\begin{aligned} 1\text{E}-9 x_1 + 1\text{E}9 x_2 &= 1 \\ 1\text{E}9 x_1 + 2\text{E}9 x_2 &= 1, \end{aligned}$$

without using any pivoting. Check the answer mentally to see if it is approximately correct. If it is, increase the exponent from 9 until a system is obtained in which the solution without pivoting is erroneous. [Note: $1\text{E}-9 = 10^{-9}$, $2\text{E}9 = 2(10^9)$]

9. Use *YUREDUCE* to solve the system in Exercise 8, which did not give a reasonable solution without pivoting, but this time use partial pivoting.
10. Repeat Exercise 9, but this time use full pivoting. Compare the answer with that in Exercise 9.
11. See how *MATCOMP* handles the linear system formed in Exercise 8.
12. Experiment with *MATCOMP*, and see if it gives a “nearly singular matrix” message when finding the inverse of a 2×2 diagonal matrix

$$rI = r \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

where r is a sufficiently small nonzero number. Use the default for roundoff control in *MATCOMP*.

3. Using MATCOMP, use the scaling routine suggested in Exercise 7 to find the inverse of the matrix

$$\begin{bmatrix} .000001 & -.000003 & .000011 & .000006 \\ -.000002 & .000013 & .000007 & .000010 \\ .000009 & -.000011 & 0 & -.000005 \\ .000014 & -.000008 & -.000002 & .000003 \end{bmatrix}$$

Then see if the same answer is obtained without using the scaling routine. Check the inverse in each case by multiplying by the original matrix.

4. Mark each of the following True or False.

- a. Addition and subtraction never cause any problem when executed by a computer.
- b. Given any present-day computer, one can find two positive numbers whose sum the computer will represent as the larger of the two numbers.
- c. A computer may have trouble representing as accurately as desired the sum of two numbers of extremely different magnitudes.
- d. A computer may have trouble representing as accurately as desired the sum of two numbers of essentially the same magnitude.
- e. A computer may have trouble representing as accurately as desired the sum of a and b , where a is approximately $2b$ or $-2b$.
- f. Partial pivoting handles all problems resulting from roundoff error when a linear system is being solved by the Gauss method.
- g. Full pivoting handles roundoff error problems better than partial pivoting, but it is generally not used because of the extra computer time required to implement it.
- h. Given any present-day computer, one can find a system of two equations in two unknowns that the computer cannot solve accurately using the Gauss method with back substitution.
- i. A linear system of two equations in two unknowns is unstable if the lines represented by the two equations are extremely close to being parallel.

- j. The entry in the i th row and j th column of a Hilbert matrix is $1/(i+j)$.



Use the routine HILBERT in LINTEK in Exercises 15–22. Because the Hilbert matrices are nonsingular, diagonal entries computed during the elimination should never be zero. Except in Exercise 22, enter 0 for r when r is requested during a run of HILBERT.

15. Solve $H_2x = b$, c , where

$$b = \begin{bmatrix} 2 \\ 9 \end{bmatrix} \quad \text{and} \quad c = \begin{bmatrix} 3 \\ 8 \end{bmatrix}.$$

Use just one run of HILBERT; that is, solve both systems at once. Notice that the components of b and c differ by just 1. Find the difference in the components of the two solution vectors.

16. Repeat Exercise 15, changing the coefficient matrix to H_2^4 .

17. Repeat Exercise 15, using as coefficient matrix H_4 with

$$b = \begin{bmatrix} 2 \\ 1 \\ 4 \\ 6 \end{bmatrix} \quad \text{and} \quad c = \begin{bmatrix} 3 \\ 0 \\ 5 \\ 7 \end{bmatrix}.$$

18. Repeat Exercise 15, using as coefficient matrix H_4^4 .

19. Find the inverse of H_2 , and use the I menu option to test whether the computed inverse is correct.

20. Continue Exercise 19 by trying to find the inverses of H_2 raised to the powers 2, 4, 8, and 16. Was it always possible to reduce the power of the Hilbert matrix to diagonal form? If not, what happened? Why did it happen? Was scaling (1 an entry) of any help? For those cases in which reduction to diagonal form was possible, did the computed inverse seem to be reasonably accurate when tested?

21. Repeat Exercises 19 and 20, using H_4 and the various powers of it. Are problems encountered for lower powers this time?

22. We have seen that, in reducing a matrix, we may wish to instruct the computer to assign the value zero to computed entries of sufficiently small magnitude, because entries that should be zero might otherwise be left nonzero because of roundoff error. Use HILBERT to try to solve the linear system

$$H_3^2 \mathbf{x} = \mathbf{b}, \quad \text{where } \mathbf{b} = \begin{bmatrix} -3 \\ 1 \\ 4 \end{bmatrix},$$

entering as r the number .0001 when requested. Will the usual Gauss-Jordan elimination using HILBERT solve the system? Will HILBERT solve it using scal (1 an entry)?

MATLAB

The command `hilb(n)` in MATLAB creates the $n \times n$ Hilbert matrix, and the command `invhilb(n)` attempts to create its inverse. For a matrix X in MATLAB, the command `max(X)` produces a row vector whose j th entry is the maximum of the entries in the j th column of X , whereas for a row vector $\mathbf{v}$, the command `max(v)` returns the maximum of the components of $\mathbf{v}$. Thus the command `max(max(X))` returns the maximum of the entries of X . The functions `min(X)` and `min(v)` return analogous minimum values. Thus the command line

`n = 5; A = hilb(n)*invhilb(n); b = [max(max(A)) min(min(A))]`

will return the two-component row vector $\mathbf{b}$ whose first component is the maximum of the entries of A and whose second component is the minimum of those entries. Of course, if `invhilb(n)` is computed accurately, this vector should be $[1, 0]$.

M1. Enter `format long` and then enter the line

`n = 2; A = hilb(n)*invhilb(n); b = [max(max(A)) min(min(A))]`

in MATLAB. Using the up-arrow key to change the value of n , copy down the vector $\mathbf{b}$ with entries to three significant figures for the values 5, 10, 15, 20, 25, and 30 of n .

- M2.** Recall that the command `X = rand(n,n)` generates a random $n \times n$ matrix with entries between 0 and 1; the Hilbert matrices also have entries between 0 and 1. Enter the command

`X = rand(30,30); A = X*inv(X); b = [max(max(A)) min(min(A))]`

to see if MATLAB has difficulty inverting such a random 30×30 matrix. Using the up-arrow key, execute this command a total of seven times, and see if the results seem to be consistent.

A

MATHEMATICAL INDUCTION

Sometimes we want to prove that a statement about positive integers is true for all positive integers or perhaps for some finite or infinite sequence of consecutive integers. Such proofs are accomplished using *mathematical induction*. The validity of the method rests on the following axiom of the positive integers. The set of all positive integers is denoted by Z^+ .

Induction Axiom

Let S be a subset of Z^+ satisfying

1. $1 \in S$,
2. If $k \in S$, then $(k + 1) \in S$.

Then $S = Z^+$.

This axiom leads immediately to the method of mathematical induction.

Mathematical Induction

Let $P(n)$ be a statement concerning the positive integer n . Suppose that

1. $P(1)$ is true,
2. If $P(k)$ is true, then $P(k + 1)$ is true.

Then $P(n)$ is true for all $n \in Z^+$.

Most of the time, we want to show that $P(n)$ holds for all $n \in \mathbb{Z}^+$. If we wish only to show that it holds for $r, r+1, r+2, \dots, s-1, s$, then we show that $P(r)$ is true and that $P(k)$ implies $P(k+1)$ for $r \leq k \leq s-1$. Notice that r may be any integer—positive, negative, or zero.

EXAMPLE A.1 Prove the formula

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2} \quad (\text{A.1})$$

for the sum of the arithmetic progression, using mathematical induction.

SOLUTION We let $P(n)$ be the statement that formula (A.1) is true. For $n = 1$, we obtain

$$\frac{n(n+1)}{2} = \frac{1(2)}{2} = 1,$$

so $P(1)$ is true.

Suppose that $k \geq 1$ and $P(k)$ is true (our *induction hypothesis*), so

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

To show that $P(k+1)$ is true, we compute

$$\begin{aligned} 1 + 2 + \cdots + (k+1) &= (1 + 2 + \cdots + k) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) = \frac{k^2 + k + 2k + 2}{2} \\ &= \frac{k^2 + 3k + 2}{2} = \frac{(k+1)(k+2)}{2}. \end{aligned}$$

Thus, $P(k+1)$ holds, and formula (A.1) is true for all $n \in \mathbb{Z}^+$. ■

EXAMPLE A.2 Show that a set of n elements has exactly 2^n subsets for any nonnegative integer n .

SOLUTION This time we start the induction with $n = 0$. Let S be a finite set having n elements. We wish to show

$$P(n): S \text{ has } 2^n \text{ subsets.} \quad (\text{A.2})$$

If $n = 0$, then S is the empty set and has only one subset—namely, the empty set itself. Because $2^0 = 1$, we see that $P(0)$ is true.

Suppose that $P(k)$ is true. Let S have $k+1$ elements, and let one element of S be c . Then $S - \{c\}$ has k elements, and hence 2^k subsets. Now every subset of S either contains c or does not contain c . Those not containing c are subsets of $S - \{c\}$, so there are 2^k of them by the induction hypothesis. Each subset containing c consists of one of the 2^k subsets not containing c , with c adjoined. There are 2^k such subsets also. The total number of subsets of S is then

$$2^k + 2^k = 2^k(2) = 2^{k+1},$$

so $P(k+1)$ is true. Thus, $P(n)$ is true for all nonnegative integers n . ■

AMPLE A.3 Let $x \in \mathbb{R}$ with $x > -1$ and $x \neq 0$. Show that $(1+x)^n > 1+nx$ for every positive integer $n \geq 2$.

SOLUTION We let $P(n)$ be the statement

$$(1+x)^n > 1+nx. \quad (\text{A.3})$$

(Notice that $P(1)$ is false.) Then $P(2)$ is the statement $(1+x)^2 > 1+2x$. Now $(1+x)^2 = 1+2x+x^2$, and $x^2 > 0$, because $x \neq 0$. Thus, $(1+x)^2 > 1+2x$, so $P(2)$ is true.

Suppose that $P(k)$ is true, so

$$(1+x)^k > 1+kx. \quad (\text{A.4})$$

Now $1+x > 0$, because $x > -1$. Multiplying both sides of inequality (A.4) by $1+x$, we obtain

$$(1+x)^{k+1} > (1+kx)(1+x) = 1+(k+1)x+kx^2.$$

Because $kx^2 > 0$, we see that $P(k+1)$ is true. Thus $P(n)$ is true for every positive integer $n \geq 2$. ■

In a frequently used form of induction known as *complete induction*, the statement

If $P(k)$ is true, then $P(k+1)$ is true

in the second box on page A-1 is replaced by the statement

If $P(m)$ is true for $1 \leq m \leq k$, then $P(k+1)$ is true.

Again, we are trying to show that $P(k+1)$ is true, knowing that $P(k)$ is true. But if we have reached the stage of induction where $P(k)$ has been proved, we know that $P(m)$ is true for $1 \leq m \leq k$, so the strengthened hypothesis in the second statement is permissible.

AMPLE A.4 Recall that the set of all polynomials with real coefficients is denoted by P . Show that every polynomial in P of degree $n \in \mathbb{Z}^+$ either is irreducible itself or is a product of irreducible polynomials in P . (An **irreducible polynomial** is one that cannot be factored into polynomials in P all of lower degree.)

SOLUTION We will use complete induction. Let $P(n)$ be the statement that is to be proved. Clearly $P(1)$ is true, because a polynomial of degree 1 is already irreducible.

Let k be a positive integer. Our induction hypothesis is then: every polynomial in P of degree less than $k+1$ either is irreducible or can be factored into irreducible polynomials. Let $f(x)$ be a polynomial of degree $k+1$. If $f(x)$ is irreducible, we have nothing more to do. Otherwise, we may factor $f(x)$ into polynomials $g(x)$ and $h(x)$ of lower degree than $k+1$, obtaining $f(x) = g(x)h(x)$. The induction hypothesis indicates that each of $g(x)$ and $h(x)$ can be factored into irreducible polynomials, thus providing such a factorization of $f(x)$. This proves $P(k+1)$. It follows that $P(n)$ is true for all $n \in \mathbb{Z}^+$. ■

EXERCISES

1. Prove that

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

for $n \in \mathbb{Z}^+$.

2. Prove that

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

for $n \in \mathbb{Z}^+$.

3. Prove that

$$1 + 3 + 5 + \cdots + (2n-1) = n^2$$

for $n \in \mathbb{Z}^+$.

4. Prove that

$$\begin{aligned} \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \cdots + \frac{1}{n(n+1)} \\ = \frac{n}{n+1} \end{aligned}$$

for $n \in \mathbb{Z}^+$.

5. Prove by induction that if
- $a, r \in \mathbb{R}$
- and
- $r \neq 1$
- , then

$$\begin{aligned} a + ar + ar^2 + \cdots + ar^n \\ = a \frac{1 - r^{n+1}}{1 - r} \end{aligned}$$

for $n \in \mathbb{Z}^+$.

6. Find the flaw in the following argument.

We prove that any two integers i and j in $\mathbb{Z}^+$ are equal. Let

$$\max(i, j) = \begin{cases} i & \text{if } i \geq j, \\ j & \text{if } j > i. \end{cases}$$

Let $P(n)$ be the statement

$P(n)$: Whenever $\max(i, j) = n$, then $i = j$.

Notice that, if $P(n)$ is true for all positive integers n , then any two positive integers i and j are equal. We proceed to prove $P(n)$ for positive integers n by induction.

Clearly $P(1)$ is true, because, if $i, j \in \mathbb{Z}^+$ and $\max(i, j) = 1$, then $i = j = 1$.

Assume that $P(k)$ is true. Let i and j be such that $\max(i, j) = k + 1$. Then

$\max(i-1, j-1) = k$, so $i-1 = j-1$ by the induction hypothesis. Therefore, $i = j$ and $P(k+1)$ is true; so, $P(n)$ is true for all n .

7. Criticize the following argument.

Let us show that every positive integer has some interesting property. Let $P(n)$ be the statement that n has an interesting property. We use complete induction.

Of course $P(1)$ is true, because 1 is the only positive integer that equals its own square, which is surely an interesting property of 1.

Suppose that $P(m)$ is true for $1 \leq m \leq k$. If $P(k+1)$ were not true, then $k+1$ would be the smallest integer without an interesting property, which would, in itself, be an interesting property of $k+1$. So $P(k+1)$ must be true. Thus $P(n)$ is true for all $n \in \mathbb{Z}^+$.

8. We have never been able to see any flaw in part a. Try your luck with it, and then answer part b.

a. A serial killer is sentenced to be executed. He asks the judge not to let him know the day of the execution. The judge says, "I sentence you to be executed at 10 A.M. some day of this coming January, but I promise that you will not be aware that you are being executed that day until they come to get you at 8 A.M." The killer goes to his cell and proceeds to prove, as follows, that he can't be executed in January.

Let $P(n)$ be the statement that I can't be executed on January $(31 - n)$. I want to prove $P(n)$ for $0 \leq n \leq 30$. Now I can't be executed on January 31, for since that is the last day of the month and I am to be executed that month, I would know that was the day before 8 A.M., contrary to the judge's sentence. Thus $P(0)$ is true.

Suppose that $P(m)$ is true for $0 \leq m \leq k$, where $k \leq 29$. That is, suppose I can't be executed on January $(31 - k)$ through January 31. Then January $(31 - k - 1)$ must be the last possible day for execution, and I would be aware that was the day

before 8 A.M., contrary to the judge's sentence. Thus I can't be executed on January $(31 - (k + 1))$, so $P(k + 1)$ is true. Therefore, I can't be executed in January. (Of course, the serial killer was executed on January 17.)

- i. An instructor teaches a class five days a week, Monday through Friday. She tells

her class that she will give one more quiz on one day during the final full week of classes, but that the students will not know for sure that the quiz will be that day until they come to the classroom. What is the last day of the week on which she can give the quiz in order to satisfy these conditions?

B

TWO DEFERRED PROOFS

PROOF OF THEOREM 4.2 ON EXPANSION BY MINORS

Our demonstration of the various properties of determinants in Section 4.2 depended on our ability to compute a determinant by expanding it by minors on *any row or column*, as stated in Theorem 4.2. In order to prove Theorem 4.2, we will need to look more closely at the form of the terms that appear in an expanded determinant.

Determinants of orders 2 and 3 can be written as

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = (1)(a_{11}a_{22}) + (-1)(a_{12}a_{21})$$

and

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = (1)(a_{11}a_{22}a_{33}) + (-1)(a_{11}a_{23}a_{32}) + (1)(a_{12}a_{23}a_{31}) + (-1)(a_{12}a_{21}a_{33}) \\ + (1)(a_{13}a_{21}a_{32}) + (-1)(a_{13}a_{22}a_{31}).$$

Notice that each determinant appears as a sum of products, each with an associated sign given by (1) or (-1), which is determined by the formula $(-1)^{i+j}$ as we expand the determinant across the first row. Furthermore, each product contains exactly one factor from each row and exactly one factor from each column of the matrix. That is, the row indices in each product run through all row numbers, and the column indices run through all column numbers. This is an illustration of a general theorem, which we now prove by induction.